
Estrutura Dados

C - Básico

Prof. Fábio Medeiros Faria
serjava.blogspot.com.br
fabio.faria@fatec.sp.gov.br

Linguagens de programação

- Linguagem de Máquina
 - Computador entende apenas pulsos elétricos
 - Presença ou não de pulso
 - 1 ou 0
- Tudo no computador deve ser descrito em termos de 1's ou 0's (binário)
 - Difícil para humanos ler ou escrever
 - $00011110 = 30$

Linguagens de programação

- Linguagem *Assembly*
 - Uso de mnemônicos
 - Conjunto de 0's e 1's é agora representado por um código
 - 10011011 -> ADD
- Linguagem *Assembly* - Problemas
 - Requer programação especial (*Assembly*)
 - Conjunto de instruções varia com o computador (processador)
 - Ainda é muito difícil programar

Linguagens de programação

- Linguagens de Alto Nível
 - Programas são escritos utilizando uma linguagem parecida com a linguagem humana
 - Independente da arquitetura do computador
 - Mais fácil programar
 - Uso de compiladores

Linguagens de programação

- Primórdios
 - Uso da computação para cálculos de fórmulas
 - Fórmulas eram traduzidas para linguagem de máquinas
 - Por que não escrever programas parecidos com as fórmulas que se deseja computar?

Linguagens de programação

- FORTRAN (FORmula TRANsform)
 - Em 1950, um grupo de programadores da IBM liderados por John Backus produz a versão inicial da linguagem;
 - Primeira linguagem de alto nível;
- Várias outras linguagens de alto nível foram criadas
 - Algol-60, Cobol, Pascal, etc

Linguagem C

- Uma das mais bem sucedidas foi uma linguagem chamada C
 - Criada em 1972 nos laboratórios por Dennis Ritchie
 - Revisada e padronizada pela ANSI em 1989
 - ANSI: American National Standards Institute
 - Padrão mais utilizado

Primeiro programa em C

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    printf("Hello World \n");

    system("pause");

    return 0;

}
```


Primeiro programa em C

```
#include <stdio.h>
#include <stdlib.h>
```

Início

```
int main() {
```

```
printf("Hello World \n");
```

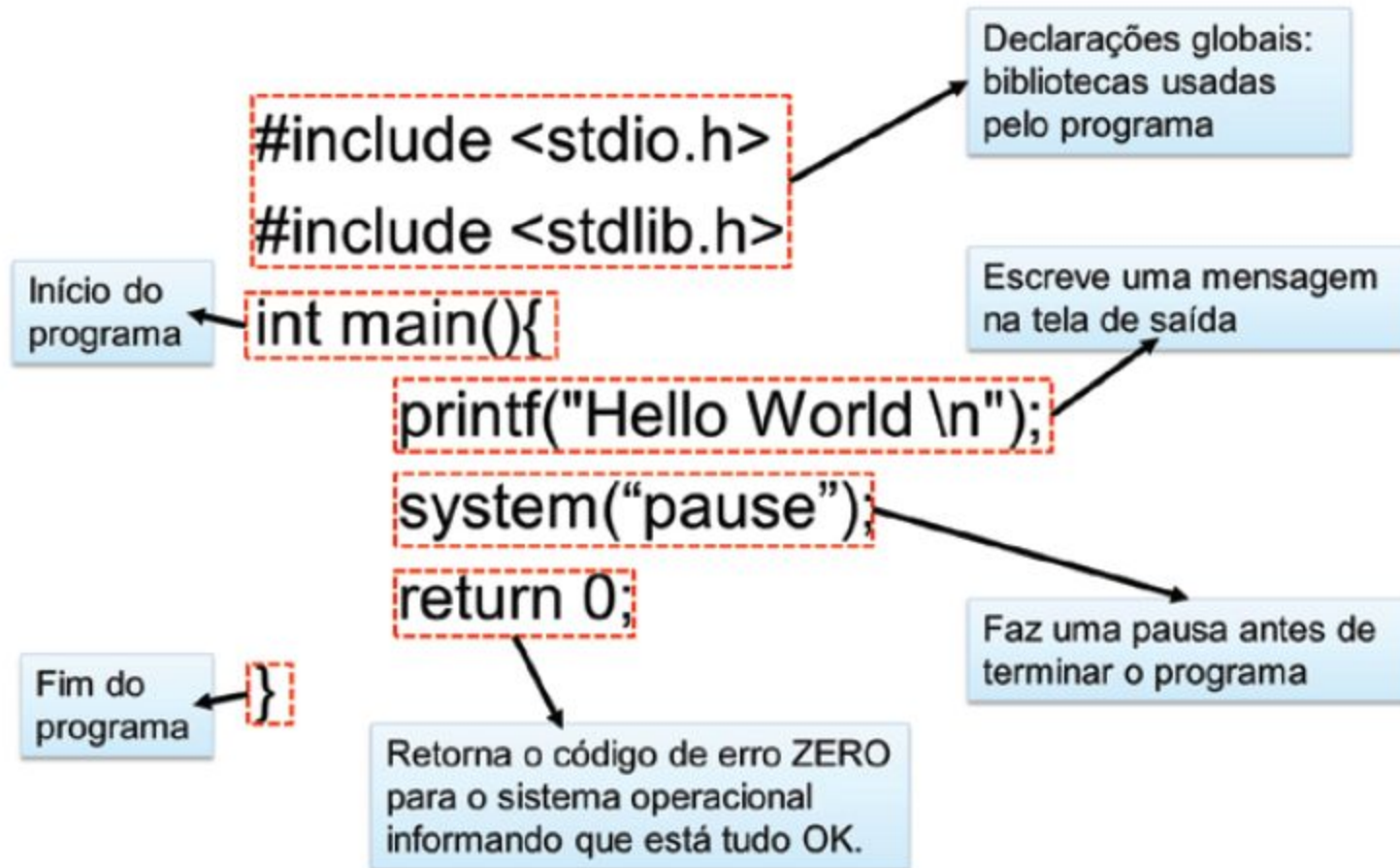
```
system("pause");
```

```
return 0;
```

```
}
```

Fim

Primeiro programa em C



Primeiro programa em C

- Por que escrevemos programas?
 - Temos dados ou informações que precisam ser processados;
 - Esse processamento pode ser algum cálculo ou pesquisa sobre os dados de entrada;
 - Desse processamento, esperamos obter alguns resultados (Saídas);

Comentários

- Permitem adicionar uma descrição sobre o programa. São ignorados pelo compilador.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    /*
    A função printf serve
    para escrever na tela
    */
    printf("Hello World \n");

    //faz uma pausa no programa
    system("pause");

    return 0;
}
```

Variáveis

- Matemática
 - é uma entidade capaz de representar um valor ou expressão;
 - pode representar um número ou um conjunto de números
 - $f(x) = x^2$

Variáveis

- Computação
 - Posição de memória que armazena uma informação
 - Pode ser modificada pelo programa
 - Deve ser **definida** antes de ser usada

Declaração de variáveis

- Precisamos informar ao programa quais dados queremos armazenar
- Precisamos também informar o que são esses dados (qual o tipo de dado)
 - Um nome de uma pessoa
 - Uma cadeia de caracteres ("André" - 5 caracteres)
 - O valor da temperatura atual
 - Um valor numérico (com casas decimais)
 - A quantidade de alunos em uma sala de aula
 - Um valor numérico (número inteiro positivo ou zero)
 - Se um assento de uma aeronave está ocupado
 - Um valor lógico (ocupado: verdadeiro / desocupado: falso)

Variáveis

- Declaração de variáveis em C
 - <tipo de dado> nome-da-variável;
- Propriedades
 - Nome
 - Pode ter um ou mais caracteres
 - Nem tudo pode ser usado como nome
 - Tipo
 - Conjunto de valores aceitos
 - Escopo
 - global ou local

Variáveis

- Nome
 - Deve iniciar com letras ou underscore (_);
 - Caracteres devem ser letras, números ou underscores;
 - Palavras chaves não podem ser usadas como nomes;
 - Letras maiúsculas e minúsculas são consideradas diferentes

Variáveis

- Nome
 - Não utilizar espaços nos nomes
 - Exemplo: nome do aluno, temperatura do sensor,
 - Não utilizar acentos ou símbolos
 - Exemplos: garça, tripé, o, Θ
 - Não inicializar o nome da variável com números
 - Exemplos: 1A, 52, 5^a
 - Underscore pode ser usado
 - Exemplo: nome_do_aluno : caracter
 - Não pode haver duas variáveis com o mesmo nome

Variáveis

- Lista de palavras chaves

auto	break	case	char	const	continue	do	double
else	for	int	union	static	default	void	return
enum	goto	long	unsigned	struct	extern	while	sizeof
float	if	short	volatile	switch	register	typeof	

Variáveis

- Quais nomes de variáveis estão corretos:
 - Contador
 - contador1
 - comp!
 - .var
 - Teste_123
 - _teste
 - int
 - int1
 - 1contador
 - -x
 - Teste-123
 - x&

Variáveis

- Corretos:
 - Contador, contador1, Teste_123, _teste, int1
- Errados
 - comp!, .var, int, 1contador, -x, Teste-123, x&

Variáveis

- Tipo
 - Define os valores que ela pode assumir e as operações que podem ser realizadas com ela
- Exemplo
 - tipo **int** recebe apenas valores inteiros
 - tipo **float** armazena apenas valores reais

Tipos básicos em C

- **char**: um byte que armazena o código de um caractere do conjunto de caracteres local
 - *caracteres sempre ficam entre 'aspas simples'!*

```
char sexo; // pode receber 'M' ou 'F'  
char UnidadeTemperatura; //pode receber 'C' para Celsius  
                        //ou 'F' para Fahrenheit
```

- **char** opcoes; // pode ser '1', '2' , '3' ou '4'
ou 32 bits

```
int NumeroAlunos;  
int Idade;  
int NumeroContaCorrente;  
int N = 10; // o variável N recebe o valor 10
```

Tipos básicos em C

- Números reais
 - Tipos: *float*, *double* e *long double*
 - A parte decimal usa **ponto** e **não vírgula!**
 - **float**: um número real com precisão simples

```
float Temperatura; // por exemplo, 23.30
float MediaNotas; // por exemplo, 7.98
float TempoTotal; // por exemplo, 0.0000000032 (s)
```

- Números muito grandes ou muito pequenos

```
double DistanciaGalaxias; // número muito grande
double MassaMolecular; // em Kg, número muito pequeno
double BalancoEmpresa; // valores financeiros
```


Tipos básicos em C

- Números reais
 - Pode-se escrever números reais usando notação científica

```
double TempoTotal = 0.000000003295;
```

```
// notação científica
```

```
double TempoTotal = 3.2950e-009;  equivale à  $3,295 \times 10^{-9}$ 
```

Variáveis

Tipo	Bits	Intervalo de valores
char	8	-128 A 127
unsigned char	8	0 A 255
signed char	8	-128 A 127
int	32	-2.147.483.648 A 2.147.483.647
unsigned int	32	0 A 4.294.967.295
signed int	32	-32.768 A 32.767
short int	16	-32.768 A 32.767
unsigned short int	16	0 A 65.535
signed short int	16	-32.768 A 32.767
long int	32	-2.147.483.648 A 2.147.483.647
unsigned long int	32	0 A 4.294.967.295
signed long int	32	-2.147.483.648 A 2.147.483.647
float	32	1,175494E-038 A 3,402823E+038
double	64	2,225074E-308 A 1,797693E+308
long double	96	3,4E-4932 A 3,4E+4932

Atribuição

- Operador de Atribuição: =
- nome_da_variável = expressão, valor ou constante;



O operador de atribuição "=" armazena o valor ou resultado de uma expressão contida à sua **direita** na variável especificada à sua **esquerda**.

- Ex.:

```
int main( ){  
    int x = 5; // x recebe 5  
    int y;  
    y = x + 3; // y recebe x mais 3
```

- A lingu
 ■ x = ; }

```
return 0;
```

Comando de saída

▣ printf()

- *print formatted*
- Comando que realiza a impressão dos dados do programa na tela

○ O t



← printf("texto");

```
#include <stdio.h>
#include <stdlib.h>
```

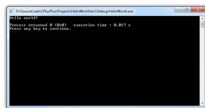
```
int main() {
    printf("Esse texto sera escrito na tela");

    return 0;
}
```

Comando de saída

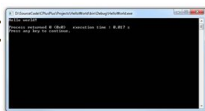
▣ printf()

- Quando queremos escrever dados formatados na tela usamos a forma geral da função, a qual possui os tipos de saída.
- Eles especificam o formato de saída dos dados que serão escritos pela função **printf()**.

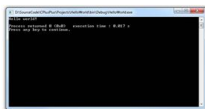


← printf("%tipo_de_saida" expressão);

- Pod



← printf("%tipo1 %tipo2", expressão1, expressão2);



← printf("texto %tipo_de_saida texto" expressão);

Comando de saída

▣ printf()

- Especificadores de formato

Alguns tipos de saída	
%c	escrita de um caractere (char)
%d ou %i	escrita de números inteiros (int ou char)
%u	escrita de números inteiros sem sinal (unsigned)
%f	escrita de número reais (float ou double)
%s	escrita de vários caracteres
%p	escrita de um endereço de memória
%e ou %E	escrita em notação científica

Comando de saída

▣ printf()

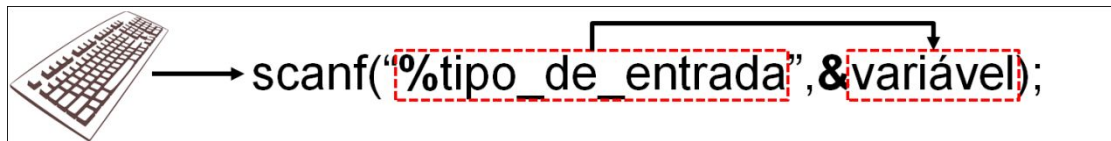
- Exemplos

```
int main() {  
    printf("Esse texto sera escrito na tela");  
  
    int x = 10;  
    float y = 20;  
    printf("%d", x);  
  
    printf("%d %f", x, y);  
  
    printf("Valor de x eh %d e o de y eh %f", x, y);  
  
    return 0;  
}
```

Comando de entrada

▣ scanf()

- Comando que realiza a leitura dos dados da entrada padrão (no caso o teclado)
- `scanf("tipo de entrada", lista de variáveis)`



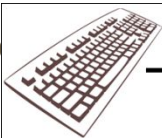
- O tipo de entrada deve ser sempre definido entre **"aspas duplas"**
- Na linguagem C, é necessário colocar o símbolo & antes do nome de cada variável a ser lida pelo comando **scanf()**.
 - O símbolo & indica qual é o endereço da variável que vai receber os dados lidos

Comando de entrada

▣ scanf()

- Comando que realiza a leitura dos dados da entrada padrão (no caso o teclado)
- `scanf("tipo de entrada", lista de variáveis)`

- O tipo



→ `scanf("%tipo_de_entrada", &variável);`

```
int main() {  
    int x;  
    scanf("%d", &x);  
  
    printf("Valor de x: %d", x);  
  
    return 0;  
}
```

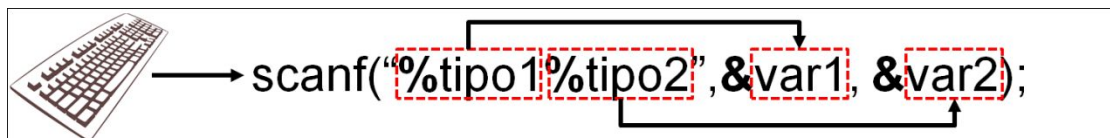
Comando de entrada

▣ scanf()

- Especificadores de formato do tipo de entrada

Alguns tipos de saída	
%c	leitura de um caractere (char)
%d ou %i	leitura de números inteiros (int ou char)
%f	leitura de número reais (float ou double)
%s	leitura de vários caracteres

- Quando digitar vários valores, separar com espaço, TAB, ou Enter



Comando de entrada

▣ scanf()

```
int main() {  
    int x, z;  
    float y;  
    //Leitura de um valor inteiro  
    scanf("%d", &x);  
    //Leitura de um valor real  
    scanf("%f", &y);  
    //Leitura de um valor inteiro e outro real  
    scanf("%d%f", &x, &y);  
    //Leitura de dois valores inteiros  
    scanf("%d%d", &x, &z);  
    //Leitura de dois valores inteiros com espaço  
    scanf("%d %d", &x, &z);  
  
    return 0;  
}
```

Comando de entrada

- **getchar()**
 - Comando que realiza a leitura de um único caractere

```
int main() {  
    char c;  
    c = getchar();  
    printf("Caractere: %c\n", c);  
    printf("Codigo ASCII: %d\n", c);  
  
    return 0;  
}
```

Escopo de variáveis

- Escopo
 - Define onde e quando a variável pode ser usada.
- Escopo global
 - Fora de qualquer definição de função
 - Tempo de vida é o tempo de execução do programa
- Escopo local
 - Bloco ou função

Escopo de variáveis

- Escopo local
 - Bloco: visível apenas no interior de um bloco de comandos
 - Função: declarada na lista de parâmetros da função ou definida dentro da função

```
//bloco
if(x == 10) {
    int i;
    i = i + 1;
}

//função
int soma (int x, int y) {
    int z;
    z = x + y;
    return z;
}
```

Escopo de variáveis

```
#include <stdio.h>
#include <stdlib.h>
void func1(){
    int x;//variável local
}
void func2(){
    int x;//variável local
}
int main(){
    int x;
    scanf("%d",&x);
    if(x == 5){
        int y=1;
        printf("%d\n",y);
    }
    system("pause");
    return 0;
}
```

- Escopo global
- Escopo local
- Escopo local dentro de outro escopo local

Constantes

- Como uma variável, uma constante também armazena um valor na memória do computador.
- Entretanto, esse valor não pode ser alterado: é constante.
- Para constantes é obrigatória a atribuição do valor.

Constantes

- Usando **#define**

- Você deverá incluir a diretiva de pré-processador **#define** antes de início do código:
- **Cuidado: não colocar ";"**

```
#define PI 3.1415
```

- Usando **const**

- Usando **const**, a declaração não precisa estar no início do código
- A declaração é igual a de uma variável inicializada

```
const double pi = 3.1415;
```

Sequências de escape

- São constantes predefinidas
- Elas permitem o envio de caracteres de controle não gráficos para dispositivos de saída

Código	Comando
\a	som de alerta (bip)
\b	retrocesso (backspace)
\n	nova linha (new line)
\r	retorno de carro (carriage return)
\v	tabulação vertical
\t	tabulação horizontal
\'	apóstrofe
\"	aspa
\\	barra invertida (backslash)
\f	alimentação de folha (form feed)
\?	símbolo de interrogação
\0	caractere nulo (cancela a escrita do restante)

Sequências de escape

- Exemplo

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    printf("Hello World\n");
    printf("Hello\nWorld\n");
    printf("Hello \\ World\n");
    printf("\\\"Hello World\\\"\\n");

    return 0;
}
```

- Saída

```
Hello World
Hello
World
Hello \ World
"Hello World"
```

Tipos Booleanos em C

- Um tipo booleano pode assumir dois valores:
 - verdadeiro ou falso (true ou false)
- Na linguagem C não existe o tipo de dado booleano. Para armazenar esse tipo de informação, use-se uma variável do tipo **int** (número inteiro)
 - Valor 0 significa falso
 - Números + ou - : verdadeiro
- Exemplos:

```
int AssentoOcupado = 1; // verdadeiro
int PortaAberta = 0; // falso
```

Operadores

- Os operadores são usados para desenvolver diferentes tipos de operações. Com eles podemos:
 - Realizar operações matemáticas com suas variáveis.
 - Realizar operações de comparação entre suas variáveis.
 - Realizar operações lógicas entre suas variáveis.
 - Realizar operações em nível de bits com suas variáveis

Operadores aritméticos

- São aqueles que operam sobre números (valores, variáveis, constantes ou chamadas de funções) e/ou expressões e têm como resultados valores numéricos
 - Note que os operadores aritméticos são sempre usados em conjunto com o operador de atribuição.

Operador	Significado	Exemplo
+	Adição de dois valores	$z = x + y$
-	Subtração de dois valores	$z = x - y$
*	Multiplicação de dois valores	$z = x * y$
/	Quociente de dois valores	$z = x / y$
%	Resto de uma divisão	$z = x \% y$

Operadores aritméticos

- Podemos devolver o resultado para uma outra variável ou para um outro comando ou função que espere receber um valor do mesmo tipo do resultado da operação, no caso, a função printf()

```
int main() {  
    int x = 10, y = 20, z;  
  
    z = x * y;  
    printf("z = %d\n", z);  
  
    z = y / 10;  
    printf("z = %d\n", z);  
  
    printf("x+y = %d\n", x+y);  
  
    return 0;  
}
```

Operadores aritméticos

- IMPORTANTE

- As operações de multiplicação, divisão e resto são executadas antes das operações de adição e subtração. Para forçar uma operação a ser executada antes das demais, ela é colocada entre parênteses
 - $z = x * y + 10;$
 - $z = x * (y + 10);$
- O operador de subtração também pode ser utilizado para inverter o sinal de um número
 - $x = -y;$
- Neste caso, a variável **x** receberá o valor de **y** multiplicado por **-1**, ou seja,
 - $x = (-1) * y;$

Operadores aritméticos

- IMPORTANTE

- Em uma operação utilizando o operador de quociente /, se o numerador e o denominador forem números inteiros, por padrão o compilador retornará apenas a parte inteira da divisão

```
int main() {  
    float x;  
    x = 5/4; // x = 1.000000  
    printf("x = %f\n", x);  
  
    x = 5/4.0; // x = 1.250000  
    printf("x = %f\n", x);  
  
    return 0;  
}
```

Operadores relacionais

- São aqueles que verificam a magnitude (qual é maior ou menor) e/ou igualdade entre dois valores e/ou expressões.
 - Os operadores relacionais são operadores de comparação de valores
 - Retorna **verdadeiro** (1) ou **falso** (0)

Operador	Significado	Exemplo
>	Maior do que	$X > 5$
>=	Maior ou igual a	$X \geq Y$
<	Menor do que	$X < 5$
<=	Menor ou igual a	$X \leq Z$
==	Igual a	$X == 0$
!=	Diferente de	$X != Y$

Importante

- Símbolo de atribuição = é diferente, muito diferente, do operador relacional de igualdade ==

```
int Nota;  
Nota == 60; // Nota é igual a 60?  
Nota = 50; // Nota recebe 50  
// Erro comum em C:  
// Teste se a nota é 60  
// Sempre entra na condição  
if (Nota = 60) {  
    printf("Você passou raspando!!");  
}  
// Versão Correta  
if (Nota == 60) {  
    printf("Você passou raspando!!");  
}
```

Importante

- Símbolo de atribuição = é diferente, muito diferente, do operador relacional de igualdade ==
- Por que sempre entra na condição?

```
if (Nota = 60) {  
    printf("Você passou raspando!!");  
}
```

- Ao fazer **No** , Um valor inteiro à variável Nota.
- O valor atribuído **60 é diferente de Zero**. Como em C os booleanos são números inteiros, então vendo **Nota** como booleano, essa assume **true**, uma vez que é diferente de zero

Operadores lógicos

- Certas situações não podem ser modeladas utilizando apenas os operadores aritméticos e/ou relacionais
 - Um exemplo bastante simples disso é saber se determinada variável x está dentro de uma faixa de valores.
 - Por exemplo, a expressão matemática
 - **$0 < x < 10$**
 - indica que o valor de x deve ser maior do que 0 (zero) e também menor do que 10

Operadores lógicos

- Os operadores lógicos permitem representar situações lógicas unindo duas ou mais expressões relacionais simples em uma composta
 - Retorna **verdadeiro** (1) ou **falso** (0)
- Exemplo
 - A expressão **$0 < x < 10$**
 - Equivale a **$(x > 0) \ \&\& \ (x < 10)$**

Operador	Significado	Exemplo
&&	Operador E	$(x > 0) \ \&\& \ (x < 10)$
	Operador OU	$(a == 'F') \ \ (b != 32)$
!	Operador NEGAÇÃO	$!(x == 10)$

Operadores lógicos

- Tabela verdade
 - Os termos ***a*** e ***b*** representam o resultado de duas expressões relacionais

a	b	!a	!b	a && b	a b
0	0	1	1	0	0
0	1	1	0	0	1
1	0	0	1	0	1
1	1	0	0	1	1

Operadores lógicos

- Exemplos

```
int main() {  
    int r, x = 5, y = 3;  
    r = (x > 2) && (y < x); //verdadeiro (1)  
    printf("Resultado: %d\n", r);  
    r = (x%2==0) && (y > 0); //falso (0)  
    printf("Resultado: %d\n", r);  
    r = (x > 2) || (y > x); //verdadeiro (1)  
    printf("Resultado: %d\n", r);  
    r = (x%2==0) || (y < 0); //falso (0)  
    printf("Resultado: %d\n", r);  
    r = !(x > 2); // falso (0)  
    printf("Resultado: %d\n", r);  
    r = !(x > 7) && (x > y); // verdadeiro (1)  
    printf("Resultado: %d\n", r);  
  
    return 0;  
}
```


Operadores de pré e pós-incremento/decremento

- Esses operadores podem ser utilizados sempre que for necessário somar uma unidade (incremento) ou subtrair uma unidade (decremento) a determinado valor

Operador	Significado	Exemplo	Resultado
++	incremento	++x ou x++	$x = x + 1$
--	decremento	--x ou x--	$x = x - 1$

Operadores de pré e pós-incremento/decremento

- Qual a diferença em usar antes ou depois da variável?

Operador	Significado	Resultado
<code>++x</code>	pré-incremento	soma +1 à variável x antes de utilizar seu valor
<code>x++</code>	pós-incremento	soma +1 à variável x depois de utilizar seu valor
<code>--x</code>	pré-decremento	subtrai -1 da variável x antes de utilizar seu valor
<code>x--</code>	pós-decremento	subtrai -1 da variável x depois de utilizar seu valor

diferença entre os dois operadores será evidente

Operadores de pré e pós-incremento/decremento

- Essa diferença de sintaxe no uso do operador não tem importância se o operador for usado sozinho
 - Porém, se utilizado dentro de uma expressão aritmética, a diferença entre os dois operadores será evidente

```
int main() {  
    int x, y;  
    x = 10;  
    y = x++;  
    printf("%d \n", x); // 11  
    printf("%d \n", y); // 10  
  
    y = ++x;  
    printf("%d \n", x); // 12  
    printf("%d \n", y); // 12  
  
    return 0;  
}
```

Operadores de atribuição simplificada

- Muitos operadores são sempre usados em conjunto com o operador de atribuição.
 - Para tornar essa tarefa mais simples, a linguagem C permite simplificar algumas expressões

Operador	Significado	Exemplo		
<code>+=</code>	Soma e atribui	<code>x += y</code>	igual a	<code>x = x + y</code>
<code>-=</code>	Subtrai e atribui	<code>x -= y</code>	igual a	<code>x = x - y</code>
<code>*=</code>	Multiplica e atribui	<code>x *= y</code>	igual a	<code>x = x * y</code>
<code>/=</code>	Divide e atribui o quociente	<code>x /= y</code>	igual a	<code>x = x / y</code>
<code>%=</code>	Divide e atribui o resto	<code>x %= y</code>	igual a	<code>x = x % y</code>

Operadores de atribuição simplificada

Sem operador

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int x = 10, y = 20;
    x = x + y - 10;
    printf("x = %d\n", x);
    x = x - 5;
    printf("x = %d\n", x);
    x = x * 10;
    printf("x = %d\n", x);
    x = x / 15;
    printf("x = %d\n", x);

    return 0;
}
```

Com operador

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int x = 10, y = 20;
    x += y - 10;
    printf("x = %d\n", x);
    x -= 5;
    printf("x = %d\n", x);
    x *= 10;
    printf("x = %d\n", x);
    x /= 15;
    printf("x = %d\n", x);

    return 0;
}
```

Operadores

- Exercício
 - Diga o resultado das variáveis x, y e z depois da seguinte sequência de operações:

```
int x, y, z;  
x = y = 10;  
z = ++x;  
x -= x;  
y++;  
x = x + y - (z--);
```

Operadores

- Exercício
 - Diga o resultado das variáveis x, y e z depois da seguinte sequência de operações:

```
int x, y;  
int a = 14, b = 3;  
float z;  
x = a / b;  
y = a % b;  
z = y / x;
```

Operadores

- Exercício
 - Diga se as seguintes expressões serão verdadeiras ou falsas:

```
int x = 7;  
(x > 5) || (x > 10)  
(!(x == 6) && (x >= 6))
```


Conversões de Tipos na Atribuição

- Atribuição entre tipos diferentes
 - O compilador converte automaticamente o valor do lado direito para o tipo do lado esquerdo do operador de atribuição "="
 - Pode haver perda de informação

```
int x = 65;
char ch;
float f = 25.1;
//ch recebe 8 bits menos significativos de x
//converte para a tabela ASCII
ch = x;
printf("ch = %c\n", ch); // 'A'
//x recebe parte apenas a parte inteira de f
x = f;
printf("x = %d\n", x); // 25
//f recebe valor 8 bits convertido para real
f = ch;
printf("f = %f\n", f); // 65.000000
//f recebe o valor de x
f = x;
printf("f = %f\n", f); // 25.000000
```

Modeladores (Casts)

- Um modelador é aplicado a uma expressão
- Força o resultado da expressão a ser de um tipo especificado.
 - (tipo) expressão
- Exemplo

```
float x, y, f = 65.5;  
  
x = f / 10.0;  
y = (int) (f / 10.0);  
printf("x = %f\n", x); //6.550000  
printf("y = %f\n", y); //6.000000
```

Precedência dos Operadores

MAIOR PRECEDÊNCIA	
++ --	Pré-incremento/decremento
()	Parênteses (chamada de função)
[]	Elemento de array
.	Elemento de struct
->	Conteúdo de elemento de ponteiro para struct
++ --	Pós-incremento/decremento
+ -	Adição e subtração unária
! ~	Não lógico e complemento bit a bit
(tipo)	Conversão de tipos (<i>type cast</i>)
*	Acesso ao conteúdo de ponteiro
&	Endereço de memória do elemento
sizeof	Tamanho do elemento
* / %	Multiplicação, divisão e módulo (resto)
+ -	Adição e subtração
<< >>	Deslocamento de bits à esquerda e à direita
< <=	"Menor do que" e "menor ou igual a"
> >=	"Maior do que" e "maior ou igual a"
== !=	"Igual a" e "diferente de"
&	E bit a bit
^	OU exclusivo
	OU bit a bit
&	E lógico
	OU lógico
?:	Operador ternário
=	Atribuição
+ = - =	Atribuição por adição ou subtração
* = / = % =	Atribuição por multiplicação, divisão ou módulo (resto)
<< = >> =	Atribuição por deslocamento de bits
& = ^ = =	Atribuição por operações lógicas
,	Operador vírgula
MENOR PRECEDÊNCIA	